agentic-ai-azure-week02-foundry-claims

Week 2 - Microsoft Foundry & Agent Service

Agentic AI on Azure - Enterprise Master Class | Handout (slides + notes)

Notes

Welcome. This deck covers 'agentic-ai-azure-week02-foundry-claims' - Week 2 - Microsoft Foundry & Agent Service. Frame the problem and why it matters, then walk the class through the learning goal, the enterprise use case, the architecture/flow, the key concepts, a live run in VS Code, and the architect's takeaways. The repo runs locally with no Azure, so demo it live.

Slide 2 - Learning goal

- Create a hosted agent on Foundry Agent Service
- Expose it via FastAPI, secured with Managed Identity
- Register a function tool the agent calls

Notes

[Learning goal] Walk through these talking points:
- Create a hosted agent on Foundry Agent Service
- Expose it via FastAPI, secured with Managed Identity
- Register a function tool the agent calls

Ask the class: which of these ideas is new to you?

Slide 3 - Enterprise use case

- Insurance Claims Intake Agent
- Extracts fields, validates policy, checks coverage, creates a case
- Uses a Foundry-hosted agent + a policy-lookup tool

Notes

[Enterprise use case] Walk through these talking points:
- Insurance Claims Intake Agent
- Extracts fields, validates policy, checks coverage, creates a case
- Uses a Foundry-hosted agent + a policy-lookup tool

Tip: relate this to a regulated scenario your students know.

Slide 4 - Architecture / flow

- Persistent hosted agent (agents.create_version)
- Run via Responses API with agent_reference
- function_call -> execute policy_lookup -> function_call_output
- Re-validate policy authoritatively before deciding

Notes

[Architecture / flow] Walk through these talking points:
- Persistent hosted agent (agents.create_version)
- Run via Responses API with agent_reference
- function_call -> execute policy_lookup -> function_call_output
- Re-validate policy authoritatively before deciding

Demo: trace a single request through these steps live.

Slide 5 - Key concepts

- PromptAgentDefinition + FunctionTool (azure-ai-projects v2)
- DefaultAzureCredential - no keys in code
- Mock backend mirrors the same extract->tool->decide flow

Notes

[Key concepts] Walk through these talking points:
- PromptAgentDefinition + FunctionTool (azure-ai-projects v2)
- DefaultAzureCredential - no keys in code
- Mock backend mirrors the same extract->tool->decide flow
Open the named file in the repo while you explain each point.

Slide 6 - Run it (VS Code - no Azure needed)

- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/claims/intake
- Runs in MOCK mode - no Azure/keys needed

Notes

[Run it (VS Code - no Azure needed)] Walk through these talking points:
- git clone the repo, then open it in VS Code
- python -m venv .venv -> activate it
- pip install -r requirements.txt
- uvicorn app.main:app --reload
- Open /docs and call POST /api/v1/claims/intake
- Runs in MOCK mode - no Azure/keys needed
Pause for questions before moving on.

Slide 7 - Architect's takeaways

- Hosted (managed threads/tools) vs. self-orchestrated
- Identity end-to-end: az login locally, Managed Identity in Azure
- Contracts are the trust boundary - never trust raw model output

Notes

[Architect's takeaways] Walk through these talking points:
- Hosted (managed threads/tools) vs. self-orchestrated
- Identity end-to-end: az login locally, Managed Identity in Azure
- Contracts are the trust boundary - never trust raw model output
Discussion: when would you NOT choose this approach?